



Plan de Trabajo

Remediación de Vulnerabilidades Críticas

PROYECTO

KognitoAI

AUDITORÍA BASE

25 de julio de 2026

DURACIÓN ESTIMADA

3 semanas (19 días hábiles)

ESTADO

Pendiente de inicio



Plan de Trabajo: Remediación de Vulnerabilidades Críticas

KognitoAI • Auditoría Técnica 2026-07-25
Generado: 25 de julio de 2026 • Responsable: Equipo de Desarrollo / Seguridad

RESUMEN EJECUTIVO

Este plan aborda las **8 vulnerabilidades críticas y altas** identificadas en la auditoría técnica del 25 de julio de 2026. Se estructura en **4 fases secuenciales** con criterios de aceptación medibles, responsables definidos y dependencias explícitas. La duración total estimada es de **19 días hábiles**, con una fase de contención inmediata de 2 días.

- Fase 0: Contención (2 días)
- Fase 1: Secretos y JWT (5 días)
- Fase 2: Aislamiento PTY (7 días)
- Fase 3: Verificación (5 días)

Fase	Nombre	Duración	Objetivo principal	Tareas
0	Contención Inmediata	2 días	Eliminar exposición crítica sin reestructuración	3
1	Endurecimiento de Autenticación y Secretos	5 días	Eliminar secretos por defecto y endurecer JWT	5
2	Aislamiento de Ejecución PTY	7 días	Eliminar ejecución arbitraria y contener blast radius	5
3	Verificación y Hardening	5 días	Testing, actualizaciones y cierre de hallazgos	6



FASE 0: Contención Inmediata

Duración: 2 días | **Objetivo:** Reducir la superficie de ataque en menos de 48 horas. **Criterio de éxito:** No se pueden explotar las vulnerabilidades más graves incluso con conocimiento interno.

TAREA 0.1 – ELIMINAR BYPASS DE VERIFICACIÓN JWT

Archivo	kai.py:98
Acción	Remover <code>options={"verify_signature": False}</code> del <code>jwt.decode()</code> . Utilizar la clave secreta configurada en <code>JWT_SECRET_KEY</code> para validar la firma del token.
Responsable	Backend
Esfuerzo	0.5 días
Criterio de aceptación	Un token con firma incorrecta o manipulado es rechazado con HTTP 401 Unauthorized.

Contexto de seguridad: El bypass actual en `kai.py` permite a cualquier usuario generar un token JWT válido sin conocer la clave secreta, comprometiendo toda la autenticación de la CLI.

TAREA 0.2 – DESACTIVAR DEBUG ENDPOINTS

Archivo	api/auth.py:422-476
Acción	Cambiar <code>settings.debug_mode</code> a <code>False</code> por defecto en <code>core/config.py</code> . Si se requieren en staging, proteger con IP whitelist estricta (solo 127.0.0.1 o VPN corporativa).
Responsable	Backend
Esfuerzo	0.5 días
Criterio de aceptación	Los endpoints <code>/auth/debug-token</code> , <code>/auth/emergency-token</code> y <code>/auth/clear-tokens</code> retornan 404 en producción.

TAREA 0.3 — RESTRINGIR CORS

Archivos	api/main.py:132-145 • api/auth.py:322-324
Acción	Reemplazar <code>allow_origins=["*"]</code> por lista blanca fija desde variable de entorno <code>CORS_ALLOWED_ORIGINS</code> . Remover header manual <code>Access-Control-Allow-Origin: *</code> en <code>api/auth.py</code> .
Responsable	Backend
Esfuerzo	0.5 días
Criterio de aceptación	Solo dominios en <code>CORS_ALLOWED_ORIGINS</code> pueden realizar peticiones; el carácter comodín <code>*</code> está excluido.

Nota: Estas tres tareas no tienen dependencias entre sí y pueden ejecutarse en paralelo por diferentes desarrolladores.



FASE 1: Endurecimiento de Autenticación y Secretos

Duración: 5 días | **Objetivo:** Garantizar que los secrets no tengan valores por defecto y que JWT sea válido solo con firma verificada. **Criterio de éxito:** No se puede iniciar la aplicación sin configurar secretos; los tokens son inviolables.

TAREA 1.1 – ELIMINAR SECRETS POR DEFECTO INSEGUROS

Archivo	core/config.py:213, 238, 419, 429
Acción	Reemplazar valores fallback ("default-admin-secret" , "supersecretkey" , "super-secret-db-encryption-key" , etc.) por None . Implementar validación en startup que lance ValueError si alguna variable crítica no está definida en el entorno.
Responsable	Backend
Esfuerzo	1 día
Criterio de aceptación	Al arrancar sin JWT_SECRET_KEY en el entorno, el proceso aborta con mensaje claro indicando la variable faltante.

TAREA 1.2 – SINCRONIZAR CONFIGURACIÓN DE TELEGRAM_GATEWAY

Archivo	telegram_gateway/config.py:37-40
Acción	Aplicar misma lógica de Fase 1.1: eliminar fallbacks y requerir variables de entorno. Verificar que el gateway no arranque si TELEGRAM_BOT_TOKEN o INTERNAL_API_KEY no están definidos.
Responsable	Backend
Esfuerzo	0.5 días
Criterio de aceptación	Telegram gateway falla al arrancar si los secretos no están en el entorno, igual que el API principal.

TAREA 1.3 – CORREGIR DECODEO JWT EN ONLYOFFICE

Archivo	onlyoffice.py:973, 1052
Acción	Especificar <code>algorithms=["HS256"]</code> de forma explícita, agregar <code>leeway=30</code> (segundos) para tolerancia de reloj, y validar el claim <code>exp</code> manualmente si es necesario. Asegurar que la clave secreta usada para decodificar coincida con <code>JWT_SECRET_KEY</code> .
Responsable	Backend
Esfuerzo	0.5 días
Criterio de aceptación	Tokens expirados o con algoritmo incorrecto son rechazados con error 401.

TAREA 1.4 – CORREGIR DECODEO JWT EN ROUTER DE CHAT

Archivo	kognito_chat/backend/router.py:556
Acción	Aplicar misma validación estricta que en Fase 1.3: algoritmo HS256 forzado, leeway controlado, validación de exp.
Responsable	Backend
Esfuerzo	0.5 días
Criterio de aceptación	Conexiones WebSocket rechazan tokens con algoritmo no HS256 o firma inválida.

TAREA 1.5 – IMPLEMENTAR ROTACIÓN DE JWT_SECRET_KEY

Archivo	core/config.py + nuevo módulo
Acción	Crear script de rotación que genere una nueva clave, la almacene como <code>JWT_SECRET_KEY_NEW</code> , mantenga validez de tokens antiguos por 24 horas usando <code>JWT_SECRET_KEY_OLD</code> , y luego limpie la clave anterior. Actualizar <code>core/config.py</code> para soportar múltiples claves en el decodeo.
Responsable	Backend + DevOps
Esfuerzo	2 días
Criterio de aceptación	Script <code>rotate_jwt_secret.sh</code> funciona en staging sin downtime; tokens emitidos antes de la rotación siguen siendo válidos durante 24h.

Dependencia: Fase 0 debe estar completada antes de iniciar Fase 1. Los cambios en secretos pueden dejar servicios inaccesibles si los debug endpoints aún están activos y se depende de ellos para diagnóstico.

FASE 2: Aislamiento del Entorno de Ejecución PTY

Duración: 7 días | **Objetivo:** Eliminar la capacidad de ejecutar comandos arbitrarios y contener el impacto de un administrador comprometido. **Criterio de éxito:** Un administrador malintencionado no puede ejecutar comandos fuera de una lista blanca ni exfiltrar secretos del entorno.

TAREA 2.1 — IMPLEMENTAR WHITELIST DE COMANDOS PTY

Archivo	api/terminal.py:183
Acción	Definir lista blanca estricta: <code>ls, cat, grep, ps, top, df, free, whoami, pwd, cd, echo, date, uptime, systemctl status, journalctl, tail, head, less, more, find, wc, sort, uniq, awk, sed</code> . Rechazar con HTTP 403 cualquier comando no listado. Implementar parser seguro que divida el comando en tokens y valide el primero contra la whitelist.
Responsable	Backend
Esfuerzo	2 días
Criterio de aceptación	Cualquier comando no whitelisteado retorna <code>{"error": "command not allowed"}</code> con código 403. Comandos con pipes o redirecciones son rechazados a menos que cada segmento esté en la whitelist.

TAREA 2.2 — ELIMINAR SHELL=TRUE EN PTY SESSIONS

Archivo	core/pty_sessions.py:77
Acción	Refactorizar a lista de argumentos: <code>cmd_list = ["bash", "-c", validated_command]</code> . Usar <code>subprocess.Popen(cmd_list, shell=False, ...)</code> . Aplicar mismo patrón en <code>quick_start.py:23</code> y <code>skills/user_global/terminal_executor_skill/scripts/terminal_executor_skill.py:63,238</code> .
Responsable	Backend
Esfuerzo	1 día
Criterio de aceptación	No se usa <code>shell=True</code> en ningún path de ejecución de PTY. Verificar con <code>grep -r "shell=True" core/ api/</code> .

TAREA 2.3 — AISLAR PTY EN CONTENEDOR DOCKER

Archivo	api/terminal.py + Dockerfile nuevo
Acción	Crear imagen <code>kognito-pty-runner</code> con las siguientes restricciones: • Filesystem <code>tmpfs</code> montado en <code>/tmp</code> (sin persistencia) • Root filesystem <code>--read-only</code> • <code>--cap-drop ALL</code> (sin capacidades Linux) • <code>--network none</code> o red restrictiva con solo salida a API interna • Límites: 512MB memoria, 0.5 CPU cores • Usuario no root (UID 1000) • Timeout de sesión: 30 minutos máximo
Responsable	DevOps + Backend
Esfuerzo	3 días
Criterio de aceptación	El proceso PTY corre en contenedor separado. Un <code>rm -rf /</code> no afecta el host. Verificar con prueba de penetración: intentar escribir en filesystem y exfiltrar datos.

TAREA 2.4 – LIMPIAR VARIABLES DE ENTORNO HEREDADAS

Archivo	api/terminal.py (construcción de env)
Acción	Construir diccionario <code>env</code> mínimo con solo <code>PATH</code> , <code>HOME</code> , <code>TERM</code> , <code>LANG</code> . Excluir explícitamente: <code>JWT_SECRET_KEY</code> , <code>DB_ENCRYPTION_KEY</code> , <code>DATABASE_URL</code> , <code>API_KEYS</code> , <code>SECRET_KEY</code> .
Responsable	Backend
Esfuerzo	0.5 días
Criterio de aceptación	Dentro del PTY, <code>env</code> no contiene ninguna clave secreta de KognitoAI. Verificar ejecutando <code>env</code> desde la terminal.

TAREA 2.5 – IMPLEMENTAR LOGGING DE AUDITORÍA

Archivo	core/pty_sessions.py + tabla <code>pty_audit_log</code>
Acción	Registrar en tabla <code>pty_audit_log</code> : <code>user_id</code> , <code>command</code> , <code>exit_code</code> , <code>timestamp</code> , <code>session_id</code> , <code>ip_address</code> . Implementar rotación de logs cada 7 días. Crear índice por <code>user_id</code> y <code>timestamp</code> para consultas rápidas.
Responsable	Backend
Esfuerzo	1 día
Criterio de aceptación	Todo comando PTY queda registrado con trazabilidad completa. Se puede generar reporte de actividad por usuario en menos de 5 segundos.

Dependencia: Fase 1 debe estar completada antes de iniciar Fase 2. El aislamiento PTY depende de tokens JWT seguros para autenticar al usuario que inicia la sesión.

FASE 3: Verificación, Testing y Hardening

Duración: 5 días | **Objetivo:** Validar que las remediaciones funcionan y cerrar brechas secundarias. **Criterio de éxito:** Cobertura de tests >80% en áreas modificadas; 0 vulnerabilidades críticas/altas pendientes.

TAREA 3.1 – ESCRIBIR TESTS DE PENETRACIÓN

Archivo	tests/security/test_penetration.py (nuevo)
Acción	Escribir tests automatizados que intenten: 1. Falsificar JWT sin firma válida 2. Ejecutar <code>rm -rf /</code> vía PTY 3. Arrancar la aplicación sin secretos configurados 4. Realizar peticiones CORS desde origen no autorizado 5. Decodificar token con algoritmo RS256 (no HS256) 6. Acceder a debug endpoints sin IP autorizada
Responsable	QA + Backend
Esfuerzo	2 días
Criterio de aceptación	Todos los ataques fallan con el código remediado. CI/CD bloquea cualquier regresión que reintroduzca la vulnerabilidad.

TAREA 3.2 – ACTUALIZAR DEPENDENCIAS CRÍTICAS

Archivo	requirements.txt
Acción	Fijar <code>cryptography=49.0.0</code> (actualmente 46.0.5), mantener <code>bcrypt ≥ 5.0.0</code> . Ejecutar <code>pip-compile</code> para generar <code>requirements.lock</code> con versiones exactas. Actualizar también <code>fastapi</code> , <code>uvicorn</code> y <code>sqlalchemy</code> a versiones estables verificadas.
Responsable	DevOps
Esfuerzo	1 día
Criterio de aceptación	<code>pip list</code> muestra versiones exactas sin rangos <code>≥</code> . <code>pip-audit</code> reporta 0 vulnerabilidades conocidas.

TAREA 3.3 – ELIMINAR CÓDIGO MUERTO/INSEGURO

Archivo	api/chat.py:9
Acción	Remover <code>import pickle</code> si no se usa. Si está oculto en condicionales no detectados, documentar o eliminar esa rama. <code>pickle</code> es inherentemente inseguro para datos no confiables.
Responsable	Backend
Esfuerzo	0.5 días
Criterio de aceptación	No hay imports de módulos inseguros no utilizados en el código productivo.

TAREA 3.4 – MIGRAR MD5 A SHA256 EN OPERACIONES SENSIBLES

Archivo	extensions/kai_ethno/skill/agents/archivist/agent.py:244
Acción	Cambiar <code>hashlib.md5()</code> por <code>hashlib.sha256()</code> para fingerprints de documentos y generación de IDs. Verificar otros archivos que usen MD5 y migrar todos los usos en operaciones que requieran resistencia a colisiones.
Responsable	Backend
Esfuerzo	1 día
Criterio de aceptación	No hay usos de MD5 en código productivo. <code>grep -r "md5" --include="*.py"</code> retorna solo comentarios o referencias históricas.

TAREA 3.5 – IMPLEMENTAR WATCHDOG PARA PTY HUÉRFANOS

Archivo	core/pty_sessions.py
Acción	Agregar <code>signal.alarm</code> o heartbeat que mate procesos si el cliente se desconecta abruptamente. Implementar limpieza de sesiones huérfanas en el arranque de la aplicación. Registrar sesiones activas en tabla <code>pty_sessions</code> con TTL.
Responsable	Backend
Esfuerzo	1 día
Criterio de aceptación	No hay procesos shell activos tras reinicio del servicio. Al desconectar un cliente WebSocket, el proceso PTY asociado es terminado en menos de 5 segundos.

TAREA 3.6 — AUDITORÍA FINAL DE REGRESIÓN

Archivo	Todos los archivos modificados
Acción	Ejecutar <code>bandit</code> , <code>semgrep</code> , <code>pip-audit</code> y revisión manual de todos los cambios. Verificar que no se reintrodujeron vulnerabilidades en código nuevo. Generar reporte comparativo con la auditoría inicial.
Responsable	Security
Esfuerzo	1 día
Criterio de aceptación	Reporte de 0 hallazgos críticos/altos. Todos los tests de penetración pasan. Documento de cierre de auditoría firmado por Security.

Dependencia: Fases 0, 1 y 2 deben estar completadas antes de iniciar Fase 3. Presupuestar tiempo buffer de 2 días adicionales si los tests de penetración descubren nuevas vulnerabilidades.



Matriz de Trazabilidad

Mapa completo de vulnerabilidades → fases → tareas. Actualizar estado semanalmente.

Vulnerabilidad	Fase	Tarea ID	Severidad	Estado
JWT sin verificación de firma (<code>kai.py</code>)	0	0.1	Crítica	Pendiente
Debug endpoints expuestos (<code>api/auth.py</code>)	0	0.2	Crítica	Pendiente
CORS abierto (<code>api/main.py</code>)	0	0.3	Media	Pendiente
Secretos por defecto (<code>core/config.py</code>)	1	1.1	Crítica	Pendiente
Secretos por defecto (<code>telegram_gateway</code>)	1	1.2	Crítica	Pendiente
JWT decode sin algoritmo estricto (<code>onlyoffice.py</code>)	1	1.3	Media	Pendiente
JWT decode sin algoritmo estricto (<code>router.py</code>)	1	1.4	Media	Pendiente
Whitelist comandos PTY (<code>api/terminal.py</code>)	2	2.1	Crítica	Pendiente
<code>shell=True</code> en PTY (<code>pty_sessions.py</code>)	2	2.2	Crítica	Pendiente
Aislamiento Docker PTY	2	2.3	Crítica	Pendiente
Exposición de secretos en env PTY	2	2.4	Media	Pendiente
Logging de auditoría PTY	2	2.5	Media	Pendiente
Tests de penetración	3	3.1	Verificación	Pendiente
Dependencias actualizadas	3	3.2	Verificación	Pendiente
Eliminar pickle muerto	3	3.3	Media	Pendiente
Migrar MD5 a SHA256	3	3.4	Media	Pendiente
Watchdog PTY huérfanos	3	3.5	Verificación	Pendiente
Auditoría final	3	3.6	Verificación	Pendiente



Acciones Rápidas (Hoy mismo)

Si necesitas acción inmediata antes de arrancar las fases formales, aplica estos parches locales:

1. Editar `kai.py`

Comentar o remover la línea con `verify_signature=False` . Asegurarse de que `jwt.decode()` use la clave secreta del entorno.

```
Antes: jwt.decode(token, options={"verify_signature": False})
Después: jwt.decode(token, JWT_SECRET_KEY, algorithms=["HS256"])
```

2. Editar `core/config.py`

Cambiar temporalmente los fallbacks a `os.environ["JWT_SECRET_KEY"]` (forzará error si no está definido).

```
Antes: JWT_SECRET_KEY = os.getenv("JWT_SECRET_KEY", "supersecretkey")
Después: JWT_SECRET_KEY = os.environ["JWT_SECRET_KEY"]
```

3. Editar `api/main.py`

Poner `allowed_origins = ["https://tudominio.cl"]` en lugar de `["*"]` .

```
Antes: allow_origins=["*"]
Después: allow_origins=os.getenv("CORS_ALLOWED_ORIGINS", "https://tudominio.cl").split(",")
```

Resultado esperado: Después de aplicar estos 3 cambios, la superficie de ataque se reduce drásticamente. La aplicación ya no arrancará sin secretos, no aceptará tokens sin firma y no permitirá CORS desde orígenes arbitrarios.



KPIs de Seguimiento

Métricas para medir el progreso y la eficacia del plan de remediación.

KPI	Meta	Medición	Responsable
Tiempo de remediación críticas	≤ 14 días	Diferencia entre fecha de auditoría y merge a main de Fase 2	Project Manager
Tareas completadas por fase	100%	Checklist de tareas por fase	Tech Lead
Cobertura de tests en código modificado	≥ 80%	<code>pytest --cov</code> en módulos modificados	QA
Dependencias con vulnerabilidades conocidas	0	<code>pip-audit</code> sin hallazgos	DevOps
Tiempo de respuesta a incidentes PTY	< 5 min	Tiempo desde ejecución no autorizada hasta detección en logs	Security
Comandos PTY bloqueados	100%	Comandos no whitelisteados retornan 403	Backend
Secrets expuestos en entorno PTY	0	Búsqueda de <code>JWT_SECRET_KEY</code> en sesiones PTY activas	Security

Dependencias entre Fases

Flujo secuencial obligatorio para garantizar la seguridad en cada etapa.



No se debe avanzar a Fase 2 sin completar Fase 1. El aislamiento PTY depende de tokens JWT seguros para autenticar al usuario que inicia la sesión. Si los secretos aún tienen valores por defecto, un atacante podría generar tokens válidos para bypassar la whitelist de comandos.

Fase 3 es obligatoria antes de considerar el plan cerrado. Sin tests de penetración y auditoría final, no hay garantía de que las remediaciones sean efectivas.

Riesgos y Mitigaciones

Posibles obstáculos y estrategias para mantener el plan en curso.

Riesgo	Probabilidad	Impacto	Mitigación
Cambios en <code>kai.py</code> rompen la CLI	Media	Alto	Verificar flujo de login de CLI en staging antes de merge. Mantener rama de rollback.
Eliminar fallbacks causa caídas en staging	Alta	Medio	Documentar variables de entorno obligatorias en README. Agregar script <code>check_env.py</code> de validación.
Whitelist PTY muy restrictiva frustra a usuarios	Media	Medio	Crear canal de feedback para solicitar comandos adicionales. Evaluar whitelist periódicamente.
Tests de penetración descubren nuevas vulnerabilidades	Media	Alto	Presupuestar tiempo buffer de 2 días adicionales en Fase 3. Escalamiento automático a Security si se encuentran hallazgos.
Aislamiento Docker PTY introduce latencia	Baja	Medio	Benchmarkear latencia antes y después. Si supera 100ms, evaluar alternativas como gVisor o Kata Containers.
Rotación de JWT causa downtime	Baja	Alto	Implementar rotación con período de gracia de 24h (clave anterior válida). Probar en staging con carga.



Próximos Pasos

Acciones inmediatas para iniciar el plan de remediación.

1. ASIGNAR RESPONSABLES

Designar un **Tech Lead** para supervisar las Fases 0-2 y un **Security Lead** para Fase 3. Confirmar disponibilidad de DevOps para tareas de contenedorización.

2. CREAR RAMAS DE TRABAJO

Crear ramas `feature` en el repositorio: `security/phase0-containment`, `security/phase1-secrets`, `security/phase2-pty-isolation`, `security/phase3-verification`. Configurar protección de rama con `required reviews`.

3. CONFIGURAR ENTORNO DE STAGING

Asegurar que el entorno de staging refleje la configuración de producción (incluyendo variables de entorno). Desplegar versión actual y verificar que todas las vulnerabilidades son explotables antes de empezar a remediar.

4. ESTABLECER CADENCIA DE SEGUIMIENTO

Reunión diaria de 15 minutos (daily standup) para revisar progreso. Demo de fin de fase para validar criterios de aceptación. Documento de trazabilidad actualizado semanalmente.

5. COMUNICAR A STAKEHOLDERS

Informar a líderes técnicos y producto sobre el plan, duración y riesgo de downtime. Establecer ventana de mantenimiento para Fase 2 (aislamiento PTY) que puede requerir reinicio de servicios.



■

Apéndice: Referencias y Recursos

DOCUMENTOS RELACIONADOS

- **AUDIT_REPORT.md** — Auditoría técnica completa del 17 de junio de 2025
- **AUDIT_REPORT_2025.md** — Auditoría técnica del 14 de febrero de 2025
- Auditoría Técnica de Código — KognitoAI (PDF) — Informe extenso generado el 25 de julio de 2026
- Repositorio: [gatovillano/KognitoAI](#)

HERRAMIENTAS RECOMENDADAS

Herramienta	Uso	Comando
bandit	Análisis estático de seguridad Python	<code>bandit -r core/ api/ -f json</code>
semgrep	Detección de patrones inseguros	<code>semgrep --config p/security-audit core/ api/</code>
pip-audit	Vulnerabilidades en dependencias	<code>pip-audit -r requirements.txt</code>
pytest	Tests unitarios y de penetración	<code>pytest --cov=core --cov=api</code>
grep	Búsqueda de patrones inseguros	<code>grep -r "shell=True\ pickle\ md5" --include="*.py"</code>

CHECKLIST DE CIERRE

- ☐ Todas las tareas de Fases 0-3 completadas
- ☐ Cobertura de tests $\geq$ 80% en código modificado
- ☐ `pip-audit` reporta 0 vulnerabilidades
- ☐ `bandit` y `semgrep` reportan 0 hallazgos críticos/altos
- ☐ Tests de penetración pasan exitosamente
- ☐ Documentación actualizada (README, variables de entorno)
- ☐ Equipo capacitado en nuevas restricciones PTY
- ☐ Plan de rotación de secretos documentado y probado

- ☐ Logging de auditoría PTY funcionando en producción
- ☐ Documento de cierre de auditoría firmado por Security

Plan de Trabajo — Remediación de Vulnerabilidades Críticas

KognitoAI • Generado el 25 de julio de 2026

Clasificación: Interno — Equipo de Desarrollo